Moment Search at Scale: What Building It Actually Taught Me

One video, one research paper, and one slide deck answering the same question from a single index — pushed through async queues, killed mid-ingest, and reported honestly where it still hurts.

Hari Chidambaram · July 30, 2026 · corpus: 1 video · 1 paper · 1 deck · 14 labeled queries

132.5 ms	0.929	0.788	108 / 108
accept p95	**recall@10**	**MRR@6**	**tests passing**
target < 300 ms	target ≥ 0.70	target ≥ 0.60	89 unit + 19 benchmark gates

01 THE PROBLEM & THE ASK

A single-source video demo has to become a mixed-corpus system that proves its own honesty

Papers and decks carry answers video can't. Bolting on a second system to hold them splits the evidence — so they enter the same queue, land in the same index, and get graded by a benchmark that isn't allowed to grade itself generously.

1

Multi-source ingestion

Parse, chunk, and embed PDFs and decks into the same Qdrant collections video already writes to. One manifest, one `kind` column, one lifecycle for all three.

2

Async work queues

A Postgres WFQ dispatcher admits fairly; Prefect executes what it admits. Registration returns `202` in ~130ms no matter what's backfilling behind it.

3

Cross-source retrieval

One `/ask_stream` call fuses video, paper, and deck evidence and cites a timestamp, a page, or a slide — never a locator the system invented.

4

Prove it, don't assert it

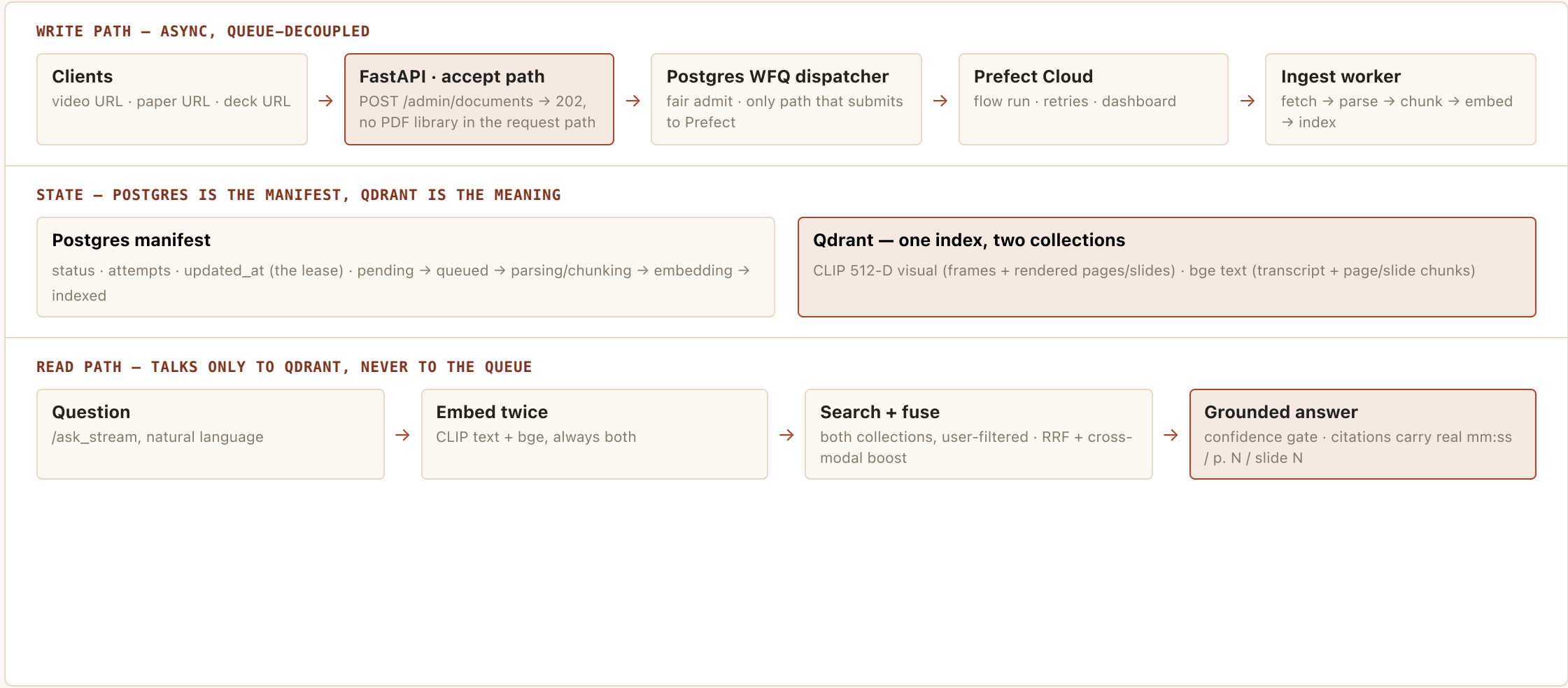
Every number in this deck traces to a script under `benchmark/` or `eval/` that anyone can re-run — including the two that failed.

> **THE BAR**
>
> Set by the assignment, not by us: **accept p95 < 300 ms · search ≤ 1.3× under backfill · recall@10 ≥ 0.70 · MRR@6 ≥ 0.60 · zero loss on a killed worker**.

02 ARCHITECTURE AT A GLANCE

One manifest, one index — search never waits on the queue

Ingestion is slow and bursty; search is fast and read-only. They share state, not a request path.



03 KEY DESIGN DECISION · 1 OF 2

We built real `.pptx` conversion — and it created a bug the tests couldn't see

Rendering PowerPoint faithfully means a LibreOffice conversion step before parsing. Getting the bytes indexed was the easy 80% of the feature; making sure a citation could still be *viewed* was the part no existing test asked about.

A

What indexing needed

- A PPTX's `storage_key` deliberately keeps pointing at the **original bytes** — a re-parse after a `PARSE_VERSION` bump always has true source material, not a lossy derivative
- Right call for ingestion; nothing in indexing or search was wrong
- Unit tests mocked storage and passed cleanly — they never asked what a browser actually receives

B

What viewing needed

- Both citation routes serve straight from `storage_key` — a PPTX-sourced citation handed the browser raw `.pptx` bytes, mislabeled `application/pdf`
- Invisible on paper: search and indexing worked perfectly; only clicking a citation was broken
- Only surfaced by actually fetching `/api/document/{id}` and checking what `file` reported

> **THE FIX**
>
> Record the converted PDF's key **once, at conversion time**, in the manifest (`view_storage_key`) — search then makes a simple choice from data it already has, instead of an object-storage existence check on every citation render.

04 KEY DESIGN DECISION · 2 OF 2

An SSRF fix that worked wasn't tight enough — so we scoped the hole to one hostname

The document fetcher has to reach one specific compose-internal address on purpose. Blocking every private range breaks that; allowing the whole private range lets an admin-token holder probe anything else on the network.

A

Pass one — real, not tight enough

- Block loopback / link-local / metadata, allow all of RFC1918 so `http://api:8000/...` keeps working
- Verified live in both directions: dangerous addresses rejected, the deck's own fetch still worked
- Still the wrong-sized exception — "needs one host" became "allow an entire private /8"

B

Pass two — scoped to the actual need

- Block every private/loopback/link-local/reserved range by default; allowlist the **one hostname** the app actually needs
- Also pin the TCP connection to the already-validated IP — closes a DNS-rebinding window where the client would otherwise re-resolve at connect time
- Re-verified both directions again after tightening, not assumed from pass one

GENERALIZES

Past SSRF: when a legitimate need punches a hole in a security boundary, scope the hole to the exact thing that needs through — **not the general category it falls under**. The gap between category and need is the residual exposure.

05 DEEP DIVE · BENCHMARK INTEGRITY

A benchmark is production code for claims — so it gets tested for false passes

The most useful test for a benchmark usually isn't the happy path. It's deliberately feeding it bad outcomes and asking whether it still turns green.

A

Caught before it shipped

- The resilience gate returned `no_loss=True` even fed already-indexed documents and two failing Docker commands — it only checked the final state
- A "recall@10" score was computed as `citations[:10]` while the API returned 6 — a label isn't a transformation
- A decoupling ratio passed after the backfill had already drained, because a `pending` row was treated as evidence of active work

B

The fix pattern

- Make overlap a hard **validity gate**, not an informational warning
- Keep the full attempted cohort in the denominator, not just the successes
- A required floor (`min_overlap_frac=0.5`) must not be loosenable from the command line down to zero

> **THE REAL BAR**
>
> A resilience proof needs four separate facts: something was **genuinely in flight**, the kill and restart **both really happened**, **nothing accepted was lost**, and a committed stage was **reused, not repeated**. "All rows are indexed now" proves none of those alone.

06 DEEP DIVE · CHASING ONE THROUGHPUT NUMBER

Four real fixes, one number that mostly didn't move — because the ceiling kept changing floors

Guessing "just add workers" would have stopped at the first plausible cause. Prefect's own run timestamps said otherwise, every time.

1

POLL INTERVAL

Flow runs sat 3–12s between `created` and `started` — Prefect's `serve()` polls Cloud every 10s by default, independent of replica count. Fix: `query_seconds=`.

2

COLD HANDSHAKES

Faster pickup meant more concurrent runs opening fresh `QdrantClient`s at once → TLS timeouts falling into Prefect's expensive 2×60s retry. Fix: short local retry, narrowly scoped.

3

THREAD OVERSUBSCRIPTION

ONNX grabs all 18 visible cores per process; N concurrent ingest subprocesses each tried to claim all of them. Fix: `TEXT_EMBED_THREADS` cap, worker-only.

4

SHARED LOCK, WRONG LAYER

Lock split + sub-batching were correct but didn't move the number — batches were too small to ever release the lock. Real fix: search's query embed runs its own local model, never through the shared service.

> **THE MOVE EACH TIME**
>
> Don't fix the mechanism the plan guessed — **timestamp the actual pipeline** (Prefect already exposes it) and ask what's one level *above* the fix just made when the metric still doesn't budge.

07 DEEP DIVE · SHARED STATE ACROSS ENVIRONMENTS

Local dev and Fly can share one queue — but not one worker's reach

One Neon manifest and one Prefect workspace on purpose, so a source registered anywhere is searchable everywhere. Nothing stopped a Fly worker from claiming a row whose bytes only exist on a laptop's disk.

A

Two admission points, not one

Suffixing deployment names by `FLY_APP_NAME` stopped a *new* run from crossing environments — but the dispatcher's own claim query still happily admitted old rows under the orphaned unsuffixed name. A `storage_env` column, checked at `db.claim_pending` — the same place every other admission invariant is checked — closed the actual race.

B

The fix revealed the next cap

Naming deployments per-environment orphaned the old shared ones, and Prefect Cloud's free-tier 5-deployment cap — undocumented anywhere in the project — then rejected registration with a `403`, silently swallowed by a bare `except: sleep 15; retry` loop.

SAME TEXT, DIFFERENT CAUSE

A `NoSuchKey` traceback from this race and an unrelated resilience FAIL looked identical — until checking one field (`STORAGE_PROVIDER`) showed only one run actually had two environments in play. **An error string names where something broke, not why.**

08 RESULTS

Retrieval quality clears every bar — throughput and error rate don't, and both are on the slide

From `benchmark/_bench.json`, the artifact `bench.py` actually writes.

Accept p95 · POST /admin/documents
target < 300 ms
132.5 ms PASS

Search p95 during backfill ÷ idle
target ≤ 1.3×
1.25× PASS

Recall@10 · 14 labeled queries
target ≥ 0.70
0.929 PASS

MRR@6 · rank-sensitive
target ≥ 0.60
0.788 PASS

Ingest throughput
target ≥ 8 chunks/s
0.96 chunks/s FAIL

Error rate · backfill cohort
target ≤ 1.0%
25.0% FAIL

Unit + benchmark self-tests
report
108 / 108

Cross-source in one answer
report
video + paper + deck

THE CAUSE

Both misses trace to one thing: a single worker replica against `DISPATCH_MAX_INFLIGHT=10` admits far more concurrent work than it can execute inside a 20-document backfill. The documented fix (`--scale worker=2`) was never applied for this measured run.

09 DEEP DIVE · RESILIENCE, RUN TWICE

A killed worker didn't lose any bytes — and still failed the resilience gate, honestly

An earlier clean pass ("10 indexed, 0 failed, 0 stuck") is recorded in this project's own history. Re-run in isolation under production defaults, it did not repeat.

A

What happened

Of 10 documents in flight, 5 were mid-chunking at kill time. After `SIGKILL` + restart, only 3 resumed cleanly from checkpoint; 2 had already-committed work **redone instead of resumed**; 2 ended `failed` on a `NoSuchKey` error that doesn't originate anywhere in this codebase.

B

What was actually true

Calling `storage.get_bytes()` by hand against the running container, for every failed document, returned the full file **immediately**. The bytes were never lost — the failure sits in the orchestration/retry layer, most likely Prefect Cloud's own post-kill task retry interacting with `RECONCILE_STALE_S`.

> SCORED ANYWAY
>
> A plain **FAIL**, not softened: the rubric's bar is "0 dropped → resumes → finished stages not re-run," and neither run met it — even though the data-loss claim doesn't hold up under direct verification. The honest report and the comfortable one are different documents.

10 THINK LIKE A FORWARD DEPLOYED ENGINEER

Every review round found something the last round's fix didn't cover — that's a pattern, not noise

1

LITERAL BEATS REASONABLE

A rule stated as a non-negotiable isn't the place to look for an implicit exception, even when the exception feels sound. Judgment applies to *how* to satisfy the rule, not *whether* it applies.

2

BOUNDARIES MEAN EVERY PATH

"Never in the request path" turned out to mean never calling the scheduler over the network, not just never importing a heavy library. A boundary named by one example is a boundary around the whole category.

3

TEST WHAT'S LITERALLY CLAIMED

"Verified live, mid-conversion crash" had only tested a crash *after* conversion completed. The stronger, literal test — an actual kill mid-conversion — found the real, narrower guarantee.

4

A FIX'S BLAST RADIUS OUTLIVES THE BUG

Removing one fallback surfaced two new bugs in sequence, each invisible from the changed file alone — visible only by asking what else in the system depended on the thing just removed.

11 WHAT SHIPS

A system whose numbers can be re-run — including the ones that aren't flattering

A

Deliverables

- **Deployed on Fly.io** — cross-source, answering the same query with video + paper + deck citations on both localhost and the public URL
- **108 / 108 tests** — 89 repository unit tests + 19 benchmark self-tests, run against the built image
- **Benchmark harness** — `bench.py`, `calibrate_thresholds.py`, `eval.py`, every figure traced to a JSON in `benchmark/`
- **14 labeled golden queries** across video, paper, and deck, each grounded in real extracted content

B

What we'd do next, in order

- **Isolate the resilience regression** — re-run with `RECONCILE_STALE_S` lowered to separate this app's checkpoint-resume from Prefect Cloud's own post-kill retry
- **Scale workers and re-measure** — `--scale worker=2` against the actual documented bottleneck, not a guess
- **A lease token at admission** — closes the duplicate-run race fully, not just its worst consequence
- **Per-corpus quota in fusion** — closes the two remaining mixed-query recall misses

> THE CLAIM
>
> Not that every gate is green. The claim is that **every number here was produced by a script anyone can run again** — and where the number is inconvenient, it's on the slide anyway.